

Poznan University of Technology
Faculty of Computing
Institute of Computing Science

Master's thesis

**INTERPRETABLE METHODS FOR DATA-TO-TEXT
GENERATION**

Dominik Maćkowiak, 151915

Supervisor
Mateusz Lango, PhD

Poznań, 2026

Abstract

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Thesis structure	1
2	Data-to-Text Generation	2
2.1	Problem definition	2
2.2	Methods	2
2.3	Interpretable D2T Approaches	2
2.4	Performance Evaluation	2
3	Building interpretable meaning representations from complex data	3
3.1	Method Overview	3
3.1.1	Conversion Methods Overview	4
3.2	Data conversion with LLMs	5
3.2.1	Text-First Extraction with Normalization	5
3.2.2	Blueprint-Iterative Refinement	6
3.2.3	Evolving Catalog	8
3.3	Fine-tuned model for data conversion	9
3.3.1	Supervised Fine-Tuning and QLoRA	9
3.3.2	Training Data Construction	10
4	Surface realization — evolving programs using AlphaEvolve	12
4.1	Method Overview	12
4.2	Modern approaches for writing complex programs	13
4.2.1	Agentic programming	13
4.2.2	Current state of the art	14
4.2.3	Challenges and possible improvements	14
4.3	AlphaEvolve	15
4.3.1	Motivation and problem formulation	15
4.3.2	Architecture of AlphaEvolve	15
4.3.3	Task specification and program representation	17
4.3.4	Prompt sampling and creative generation	17
4.3.5	Evaluation, evolutionary storage, and distributed execution	17
4.3.6	Scope and limitations of AlphaEvolve	18
4.4	OpenEvolve	18
4.4.1	Open-source implementation	18
4.4.2	Execution architecture used in the thesis	18
4.4.3	Extensions made for the experiments	19

4.5	Program database and initial program	19
4.5.1	Program records and persistence	19
4.5.2	Initial program	20
4.6	Program selection and prompt engineering	20
4.6.1	Parent and inspiration selection	20
4.6.2	Prompt composition	21
4.6.3	SEARCH/REPLACE instructions and their extensions	21
4.7	Large Language Model	22
4.7.1	Parsing the LLM response	22
4.7.2	Invalid programs and execution safety	22
4.8	Evaluation	22
4.8.1	Evaluator integration	22
4.8.2	Reference data and candidate execution	23
4.8.3	Reference-based scoring	23
4.8.4	LLM as a judge	23
4.8.5	Artifacts as evaluator feedback	23
4.8.6	Registration and continuation of the loop	24
5	Experiments	25
5.1	Meaning representation	25
5.1.1	Data Collection	25
5.1.2	Training Configuration	26
5.2	Surface realization	27
5.2.1	Illustrative evolution trace	27
5.3	Whole system evaluation	27
6	Discussion	28
7	Summary	29
	Bibliography	30
A	Prompts	31
A.1	Text-First Extraction with Normalization	31
A.2	Blueprint-Iterative Refinement	31
A.3	Evolving Catalog	32
B	Prompts for Fine-tuned Model	34

Chapter 1

Introduction

1.1 Motivation

1.2 Thesis structure

The structure of the thesis is as follows. Chapter 2 provides an overview of data-to-text generation. Chapter 3 describes the methods for building interpretable meaning representations from complex data. Chapter 4 presents the surface realization approach using OpenEvolve. Chapter 5 reports the experimental results. Chapter 6 discusses the findings, and Chapter 7 concludes the thesis.

Chapter 2

Data-to-Text Generation

2.1 Problem definition

2.2 Methods

2.3 Interpretable D2T Approaches

2.4 Performance Evaluation

Chapter 3

Building interpretable meaning representations from complex data

3.1 Method Overview

The central problem addressed in this work is the conversion of complex structured data into interpretable meaning representations. Modern data sources frequently provide information in structured formats such as JSON or XML, containing nested fields, time series, categorical values, and hierarchical relationships. While these formats are efficient for data storage and exchange, they are not directly amenable to semantic reasoning, or natural language understanding tasks. The challenge lies in extracting the semantic content from these structures and representing it in a standardized, machine-interpretable form.

The goal is to transform structured data into semantic triples—subject-predicate-object statements that capture the essential meaning of the data. For example, consider a JSON object describing a mobile phone with nested specifications:

```
{"name": "PhoneModel", "display": {"size": "6.5 inch", "resolution": "2400x1080"},  
  "battery": {"capacity": "5000 mAh"}, "camera": {"main": "108 MP"}}
```

This structured data should be converted into triples such as:

- (PhoneModel, hasDisplaySize, 6.5 inch)
- (PhoneModel, hasDisplayResolution, 2400x1080)
- (PhoneModel, hasBatteryCapacity, 5000 mAh)
- (PhoneModel, hasMainCamera, 108 MP)

These triples capture the semantic relationships in a format that can be directly used for querying, and reasoning.

This transformation presents several challenges that must be addressed by any practical method. First, the input data varies significantly in structure and complexity: some sources contain flat key-value pairs, while others have deeply nested hierarchies, time series with dozens of data points, or categorical attributes with multiple values. Second, the extraction must preserve semantic accuracy, ensuring that the generated triples faithfully represent the information in the source data without introducing errors or omissions. Third, predicate normalization is essential for consistency: the same relationship should be expressed using the same predicate across different instances (e.g., always `hasBatteryCapacity` rather than sometimes `battery_capacity`, sometimes `has_battery`,

etc.). Fourth, the methods must scale to handle large datasets efficiently, as real-world applications may involve thousands or millions of instances.

To address these challenges, two complementary approaches are investigated in this work. The first consists of pipeline-based methods that use large language models in a multi-stage process: first generating natural-language references from the structured data, then extracting semantic triples from those references. The second approach distills this behavior into a single domain-specialized fine-tuned model that produces both the natural-language text and the semantic triples in a single pass. Both approaches are described in detail in the following sections.

3.1.1 Conversion Methods Overview

Two complementary approaches are investigated for converting raw structured instances into semantic triples. The first consists of three pipeline variants that use a general-purpose LLM to first generate reference texts and then extract triples, while the second distills this behavior into a single domain-specialized fine-tuned model.

Pipeline-based methods. The first approach, described in Section 3.2, consists of three pipeline variants that convert data to triples using large language models. All three methods follow a common two-stage structure: in the first stage, each raw structured instance is converted into a natural-language reference text; in the second stage, semantic triples are extracted from that text. The three variants differ in how they manage predicate consistency and validation.

The fundamental challenge is that predicate consistency—ensuring the same semantic relationship is expressed with the same predicate across all instances—requires some form of coordination between extraction calls, but such coordination introduces sequential dependencies that reduce throughput. A fully parallel approach maximizes speed but produces inconsistent predicates; a fully sequential approach ensures consistency but is slow. The three variants explore different points in this tradeoff space:

- **Text-First Extraction with Normalization** prioritizes throughput by extracting triples independently in parallel, accepting predicate inconsistency during extraction and resolving it through a post-hoc normalization stage. This approach is the fastest but requires additional LLM calls for normalization.
- **Blueprint-Iterative Refinement** prioritizes interpretability and control by creating an explicit, human-inspectable specification (the blueprint) that defines the predicate vocabulary upfront. The blueprint is iteratively validated and refined against reference texts before extraction. This approach provides the most structured and auditable output but requires multiple validation rounds.
- **Evolving Catalog** seeks a middle ground by building predicate consistency incrementally during a sequential bootstrap phase, then switching to batch processing once the predicate vocabulary stabilizes. This hybrid approach balances consistency and throughput, adapting to the data’s complexity.

Fine-tuned model. The second approach, described in Section 3.3, distills the behavior of the multi-stage pipeline into a single domain-specialized model. Using *QLoRA* (Quantized Low-Rank Adaptation), a model is fine-tuned to produce both natural-language text and semantic triples in a single decode pass. This approach trades the flexibility of the pipeline for reduced inference

latency and improved consistency, at the cost of requiring per-domain fine-tuning on reference outputs produced by the pipeline.

3.2 Data conversion with LLMs

Pipeline overview. All three methods follow a common two-stage structure. In the first stage, each raw structured instance is converted into a natural-language reference text. In the second stage, semantic triples are extracted from that text. This text-first approach decouples verbalization from triple extraction, allowing each stage to be optimized independently and batched for throughput. The rationale for this decomposition is that natural language serves as an intermediate representation that generalizes the input data, while the structured triple output can be directly used for downstream tasks such as program synthesis.

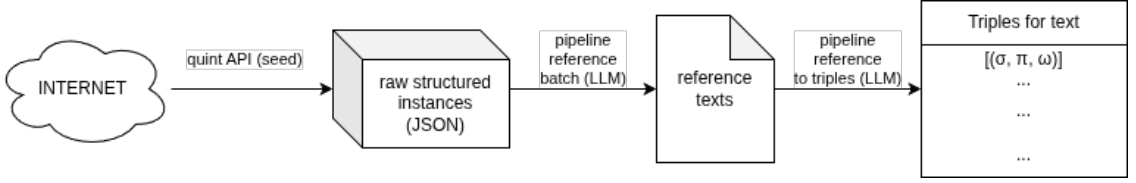


FIGURE 3.1: General pipeline flow: raw structured data from quintd is first converted to reference texts, then to semantic triples using one of the three pipeline methods.

3.2.1 Text-First Extraction with Normalization

Text-First Extraction with Normalization decomposes the data-to-triples problem into two fully batched LLM stages and a normalization stage. The method deliberately avoids any inter-instance coupling during extraction: each reference and each triple set is produced independently and in parallel, which makes the method stateless, at the cost of not enforcing predicate consistency during extraction.

Stage 1 — Reference text generation. In the first stage the model is invoked once per instance under a `text-generation system prompt`, P_{txt} (see Appendix A.1), whose objective is to convert one structured data instance into concise natural language and to return the result as a JSON object of schema `{"text": "..."}1. The prompt further instructs the model to capture the most important information, trends, extremes, and notable conditions, and to summarize time series (e.g., weather forecasts) at a high level rather than listing every data point.`

All N requests are packaged as a single OpenAI Batch API job and a single request outputs t_i , where t_i is the generated reference text for instance d_i . This batched stage yields, for every instance, a natural-language reference t_i that constitutes the intermediate representation from which triples are subsequently derived.

Stage 2 — Triples-from-text extraction. A second OpenAI Batch is then assembled in which every reference t_i is paired with a `triples-from-text system prompt`, P_{tr} (see Appendix A.1), whose objective is to extract semantic triples from natural language text, returning JSON of schema `{"triples": [{"subject": "...", "predicate": "...", "object": "..."}]}`. The prompt imposes two constraints: it instructs the model to preserve the full meaning of the input text and to keep predicates in `lower_snake_case` when possible, and to avoid duplicate triples.

¹Throughout this work, JSON schemas are presented in simplified notation with placeholder values (e.g., `"text": "..."`) for readability, rather than as formal JSON Schema definitions.

Each instance is therefore an independent text-to-triples problem, and the batch can be fully parallelized. The output of a single request is T_i , where T_i is a set of triples that describe the reference text t_i .

Stage 3 — Post-hoc predicate normalization. Because Stage 2 imposes no inter-instance vocabulary control, this stage invokes a predicate-normalization routine. The routine operates on the flattened list of triples and iteratively merges semantically equivalent predicates:

Let Π be the set of distinct predicates present in the extracted triples. A Union-Find structure over Π is initialized with every predicate in its own singleton class. A *Union-Find* (disjoint-set) is a data structure that maintains a collection of disjoint sets and supports two operations: **find**(x) returning the representative element of the set containing x , and **union**(a , b) merging the sets containing a and b . A *singleton class* is an equivalence class containing only a single element.

For each round $r = 1, 2, \dots$ (bounded by $2|\Pi|$):

- The model is prompted with the current set of canonical predicates (class representatives) and asked to propose candidate equivalent pairs under a **candidate-pairs system prompt** (see Appendix A.1).
- If no candidate pairs are returned, or if the candidate set is identical to one proposed in an earlier round (indicating the model has converged or is cycling without progress), the loop terminates.
- Otherwise, every candidate pair is sent to the model under a **strict-equivalence system prompt** (see Appendix A.1) that returns a JSON object with three fields: **equivalent** (boolean), **confidence** (a score between 0 and 1), and **reason** (a textual explanation). The model is instructed to judge equivalence strictly (mergeable in a knowledge graph), not mere relatedness.
- Pairs judged equivalent are merged in the Union-Find structure. If a round produces no merges, the loop terminates.

After termination, each predicate is mapped to the canonical representative of its class, defined as the first member of the class (deterministic, order-based choice). The triples are then rewritten with these canonical predicates.

In the normalization step, the final output combines: (i) generated references, (ii) extracted triples, and (iii) normalized triples alongside the canonical mapping and the comparison audit trail.

3.2.2 Blueprint-Iterative Refinement

Blueprint-Iterative Refinement introduces an explicit notion of extraction blueprints that act as an intermediate, human-inspectable artifact between the domain declaration and the final triple extraction. An *extraction blueprint* is a structured specification containing predicate labels, example triples, example text, and transformation guidelines. The blueprints are generated top-down from the domain name, then iteratively validated and revised against the generated references until it is confirmed that they can extract triples for every instance. Only then is a single batched triple-extraction run executed under the finalized blueprint.

Stage 1 — Initial blueprint generation. Given the user-supplied problem domain string δ (e.g., weather forecast), the model is invoked once under a **blueprint-creation system prompt**, $P_{\text{blueprint}}$ (see Appendix A.2), and asked to design an extraction blueprint for a domain. Its

response is constrained by a strict JSON schema to contain exactly four fields:

$$R = (\Pi_R, E, T_{\text{ex}}, G),$$

where Π_R is a list of predicate labels, E is a list of example triples $\{(\sigma_k, \pi_k, \omega_k)\}$ using those predicates, T_{ex} is a short natural-language passage consistent with the example triples, and G is a list of free-text transformation guidelines. The response is sanitized, which deduplicates predicates and guidelines, discards malformed example triples, and falls back to the corresponding field values from the previous blueprint R_{r-1} for any field that is empty or malformed (for the initial blueprint R_0 , empty fields default to empty lists or strings). The resulting R_0 constitutes the initial blueprint. Conceptually, R_0 is a self-contained specification of how the model believes instances of the domain should be verbalized and triplelized, complete with concrete formatting conventions embedded in the example triples.

Stage 2 — Reference text generation. The reference-generation batch is identical to that of the previous method: every instance d_i is sent to the **text-generation system prompt** P_{txt} within a single OpenAI Batch, producing references t_i .

Stage 3 — Blueprint validation and iterative revision. The core of the method is an iterative loop, bounded by R_{max} . The loop maintains a current blueprint R and operates as follows for each round $r = 1, \dots, R_{\text{max}}$:

1. **Feasibility scan.** A random reference text with replacement t_i is selected and the model is invoked under a **blueprint-feasibility system prompt**, P_{feas} (see Appendix A.2), which receives the current blueprint R and the text t_i , and returns a JSON object with three fields: **possible** (boolean indicating whether the blueprint is sufficient), **reason** (textual explanation), and **blueprint_gaps** (a list of descriptions of what is missing from the blueprint in order to extract triples from t_i).
2. **Pass condition.** If the scan reaches the R_{max} without encountering a **possible = false** judgment, the blueprints are declared validated against all texts and the loop terminates.
3. **Failure and revision.** As soon as a reference t_{i^*} is judged infeasible under R , the scan halts. The **blueprint-revision system prompt**, P_{rev} (see Appendix A.2), is then invoked with the domain δ , the current blueprint R , the failing text t_{i^*} , the failure reason, and the enumerated **blueprint_gaps**. Its response is again constrained to the same four-field schema structure $R = (\Pi_R, E, T_{\text{ex}}, G)$ described in Stage 1. The model is instructed to keep existing high-quality blueprint components, but add or adjust what is necessary to cover the failing case. The sanitized result replaces R as the new current blueprint.
4. **Restart.** Because revising the blueprint may break previously validated instances or expose new gaps, the scan is restarted and the round number r set to 1. The loop thus monotonically accumulates coverage, but backtracks the scan position whenever the blueprint is modified.

If the budget R_{max} is exhausted before all texts are validated, the method emits a warning and proceeds to final extraction with the best-effort blueprint; the output records that validation was incomplete.

This stage implements a generate-test-revise loop in which the blueprint is the artifact being refined, the reference texts act as test cases, and the feasibility prompt acts as a model-internal test oracle.

Stage 4 — Final batched extraction under finalized blueprint. Once the validation loop has terminated, a single OpenAI Batch is submitted in which every reference t_i is paired

with a **triples-from-text-with-blueprint system prompt**, $P_{\text{tr}\&R}$ (see Appendix A.2). This prompt instructs the model to extract semantic triples from the natural language text using the provided blueprint, to follow the blueprint guidelines, to mimic the formatting style of the example triples, and to prefer predicates from the blueprint predicate list whenever possible. The blueprint R is serialized and injected in each request. The batch output is parsed into T_i .

3.2.3 Evolving Catalog

Evolving Catalog is a hybrid text-first data-to-triples pipeline that combines an evolving predicate catalog with a stability-triggered transition from sequential to batch processing. It is designed to keep the predicate vocabulary consistent across instances while reducing the latency overhead of per-instance, order-dependent LLM calls.

Stage 1 — Reference text generation. An OpenAI Batch is assembled in which every instance d_i is paired with the **text-generation system prompt** P_{txt} (see Appendix A.3), whose purpose is to produce a concise natural-language summary that captures the most important information, trends, extremes, and notable conditions, and summarizes time series at a high level. The batch output is parsed into text reference t_i .

Stage 2 — Sequential catalog-driven triple extraction. The second stage maintains a mutable predicate catalog C , initialized as empty. The catalog is a set of entries $\{(\pi, e_\pi)\}$, where each entry associates a predicate label π with an example triple $e_\pi = (s, \pi, o)$ in which that predicate was first observed. The catalog thus carries both a normalized vocabulary and a small set of usage examples that serve as in-context guidance for the extraction model.

The instances are processed strictly in order:

1. **Bootstrap instance** ($i = 0$). Because C is empty, the reference t_0 is sent to the model under a **first-instance system prompt**, P_{first} (see Appendix A.3), that simply asks for semantic triples from the text with concise **lower_snake_case** predicates and no duplicates. This yields a set of triples T_0 .
2. **Catalog-guided instances** ($i \geq 1$). For each subsequent instance d_i , the reference t_i is sent under a **catalog-guided system prompt**, P_{cat} (see Appendix A.3), in which the current catalog C is serialized and injected into the user prompt. The model is instructed to use only predicates from the provided catalog when possible and to introduce a new predicate only when none of the catalog predicates can describe the fact. This yields triples T_i .

After each instance, every triple in T_i is inspected. If a triple contains a predicate $\pi \notin C$, a new entry (π, e_π) is added to C , where e_π is set to that triple (the first occurrence).

Stage 3 — Stable-window heuristic and batched tail processing. A counter w tracks the number of consecutive instances that did not introduce any new predicate; it is reset to zero whenever a new predicate is added. Let W denote the stable window parameter. The sequential phase terminates as soon as $w \geq W$ and at least one instance remains unprocessed. Let i^* be the index at which this condition is first met; the catalog C at that point is declared frozen.

All instances $d_{i^*+1}, \dots, d_N$ (the tail) are then processed in a single second OpenAI Batch, again under P_{cat} and with the frozen catalog C injected into every request. Two design choices are important here:

- The tail is parallelized, hence the cost of the remaining $N - i^* - 1$ LLM calls is amortized into one batch.

- Predicates that first appear within the tail are not added back to C . The catalog is finalized at the moment of stability, which guarantees that the predicate vocabulary presented to a human evaluator is fully determined by the sequential phase and does not silently grow during batch processing.

3.3 Fine-tuned model for data conversion

The methods described in Section 3.2 rely on repeated invocations of a general-purpose large language model at inference time: each instance requires one or more LLM calls to produce reference text, extract triples, and, in some pipelines, to validate or revise intermediate artifacts. While this design keeps the pipeline flexible, it incurs substantial latency and computational cost, particularly when the same domain is processed repeatedly. An alternative is to *distill* the behaviour of the multi-stage pipeline into a single, domain-specialized model that produces both the natural-language reference and the corresponding triple set in one decode pass. This section describes how such a model is obtained via supervised fine-tuning with QLoRA.

3.3.1 Supervised Fine-Tuning and QLoRA

Supervised fine-tuning (SFT) adapts a pre-trained language model to a specific task by training it on input–output pairs. Given a pre-trained model M_θ with parameters θ and a dataset $\{(x_i, y_i)\}_{i=1}^n$ of input–output examples, SFT minimizes the negative log-likelihood of the targets [9]:

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{t=1}^{|y_i|} \log P_\theta(y_{i,t} \mid x_i, y_{i,<t}). \quad (3.1)$$

For small models (up to a few billion parameters), full SFT—updating all parameters θ —is feasible on modern GPU hardware. However, for models with tens of billions of parameters, full fine-tuning becomes prohibitively expensive: a 31-billion-parameter model in 16-bit precision requires approximately 62 GB of memory for the weights alone, and the standard Adam optimizer [4] maintains two additional moment estimates per parameter, roughly doubling the memory footprint.

Parameter-efficient fine-tuning (PEFT) methods address this by keeping the base model weights frozen and introducing a small number of trainable parameters [7]. Among these, *Low-Rank Adaptation* (LoRA) [2] has become the most widely adopted. LoRA decomposes the weight update into a low-rank product: for each frozen weight matrix W_0 , the forward pass is modified as

$$h = W_0x + \Delta Wx = W_0x + \frac{\alpha}{r}BAx, \quad (3.2)$$

In this formulation, W_0x represents the output of the original pretrained model, while ΔWx is the learned task-specific adaptation. The key insight from Hu et al. [2] is that the weight updates ΔW during fine-tuning also have low “intrinsic rank,” meaning they can be approximated by a low-rank decomposition. Rather than updating the full weight matrix W_0 , LoRA keeps W_0 frozen and learns the low-rank matrices A and B , where $\Delta W = \frac{\alpha}{r}BA$. The scaling factor $\frac{\alpha}{r}$ controls the magnitude of the adaptation relative to the pretrained weights. This decomposition reduces the number of trainable parameters from $d \times k$ (the full weight matrix) to $r \times (d + k)$ (the two low-rank matrices), where $r \ll \min(d, k)$.

where:

- $W_0 \in \mathbb{R}^{d \times k}$ is the frozen pretrained weight matrix
- d is the output dimension and k is the input dimension of the layer

- $x \in \mathbb{R}^k$ is the input vector to the layer
- $h \in \mathbb{R}^d$ is the output (hidden state) of the layer
- $A \in \mathbb{R}^{r \times k}$ is the first trainable low-rank matrix (down-projection)
- $B \in \mathbb{R}^{d \times r}$ is the second trainable low-rank matrix (up-projection)
- $r \ll \min(d, k)$ is the rank of the decomposition
- α is a scaling factor that controls the magnitude of the low-rank update

Only A and B are updated by the optimizer; W_0 remains frozen and requires no gradient storage or optimizer state. For a typical transformer with attention projections W_q, W_k, W_v, W_o and feed-forward projections $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$, LoRA adapters are attached to each projection, yielding a trainable parameter count that is a small fraction of the base model (on the order of tens of millions versus tens of billions).

At the data scales typical for domain-specific data-to-text tasks (on the order of several hundred to a few thousand labeled examples), LoRA’s low-rank constraint also acts as a regularizer: by limiting the number of trainable parameters, it reduces the risk of overfitting and preserves the general linguistic capabilities acquired during pre-training [2].

QLoRA [1] extends LoRA with two memory-reduction techniques that together allow fine-tuning a 31-billion-parameter model on a single GPU with 40 GB of VRAM. First, the frozen base model weights are quantized to *4-bit NormalFloat* (NF4), a quantization datatype that is information-theoretically optimal for normally distributed weights. A *double quantization* step further quantizes the quantization constants themselves, reducing the per-parameter overhead. Second, *paged optimizers* leverage NVIDIA unified memory to transparently page optimizer state between GPU and CPU memory when the GPU reaches capacity, avoiding out-of-memory failures during gradient accumulation steps. The forward and backward passes are performed in `bfloat16` precision: the 4-bit weights are dequantized on the fly for each matrix multiplication, preserving numerical quality close to a full-precision fine-tune while keeping the memory footprint low.

3.3.2 Training Data Construction

Rather than collecting human-annotated data-triple pairs, the training data for the fine-tuned model is produced by *distilling* one of the pipeline described in Section 3.2. For each raw structured instance d_i processed by a tripler method, the pipeline already produces a reference text t_i and a set of normalized triples $T_i = \{(s_k, \pi_k, o_k)\}$. These two outputs, aligned by instance identifier, constitute exactly the joint target that the fine-tuned model should learn to produce in a single pass.

Each training example is formatted as a three-message conversation following the base model’s chat template:

- **System prompt.** An instruction directing the model to convert one structured data instance into concise natural language and a corresponding set of RDF semantic triples (see Appendix B for the complete prompt text), and to return the result as a JSON object of schema `{"text": "...", "triples": [{"subject": "...", "predicate": "...", "object": "..."}]}`.
- **User message.** A prompt that presents the serialized instance and requests the reference text and triples (see Appendix B for the complete template). The wording and instance

serialization are kept byte-identical to the prompts used by the tripler pipeline at inference time, ensuring *prompt parity*: the token sequence seen during training matches the token sequence the model will encounter when served.

- **Assistant message (target).** A JSON string encoding the reference text and the normalized triples (see Appendix B for the expected output format): `{"text":  $t_i$ , "triples":  $T_i$ }`.

The messages are rendered through the base tokenizer’s `chat_template` function, which applies the special tokens and formatting expected by the model architecture. This ensures that the token sequences fed to the trainer are identical to those assembled by the serving engine at inference time.

A notable consequence of this distillation approach is that the fine-tuned model is bounded by the quality of the reference labels: it can learn to reproduce the pipeline’s outputs more efficiently and consistently, but it cannot exceed their quality. Improving the reference labels is an upstream concern addressed by the tripler pipeline design, not by fine-tuning.

The dataset is split into a training, evaluation and test set. The split is deterministic, controlled by a fixed random seed, so that results are reproducible across runs. One fine-tuned model is produced per domain, keeping the predicate vocabulary and verbalization style specialized to that domain.

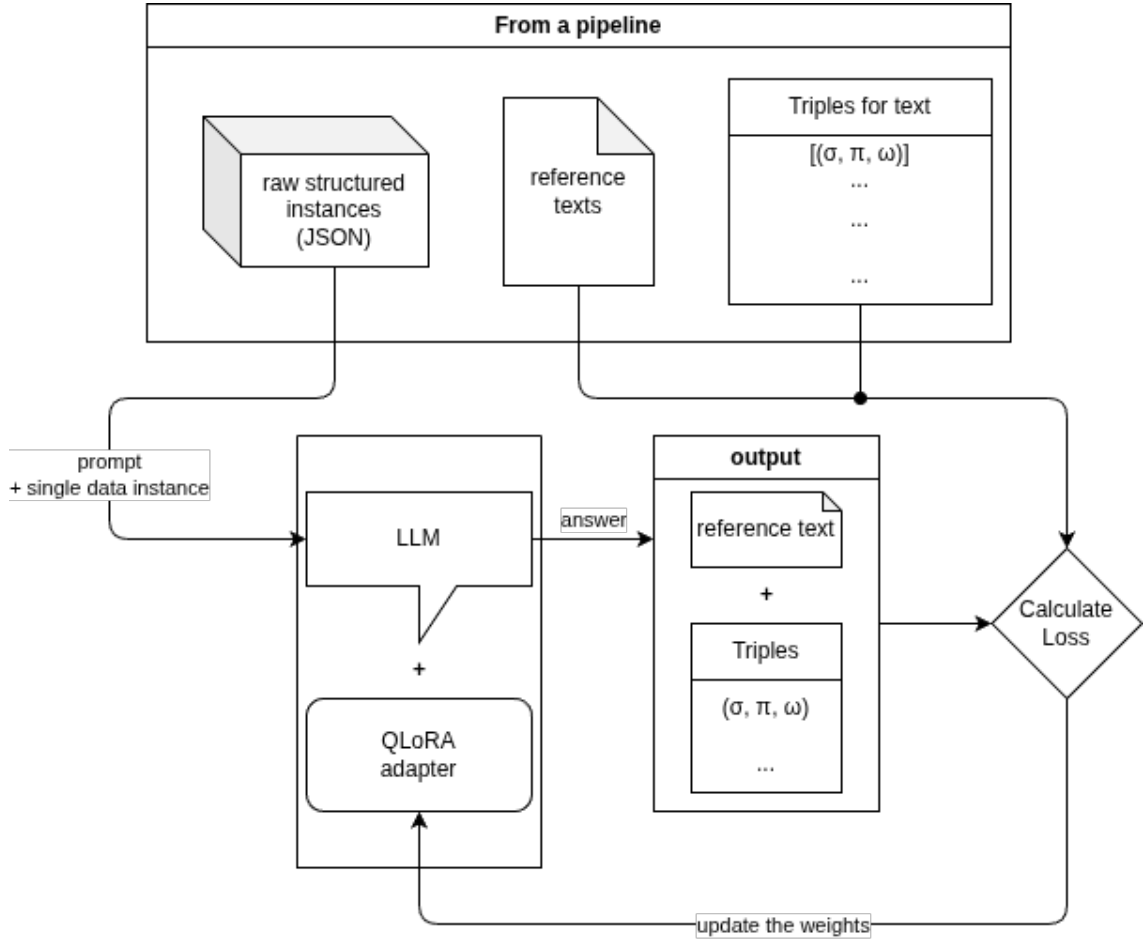


FIGURE 3.2: Fine-tuning flow: the raw structured instances, reference texts and triples are used to train a domain-specialized model via QLoRA. The resulting model can produce both outputs in a single decode pass.

Chapter 4

Surface realization — evolving programs using AlphaEvolve

4.1 Method Overview

The previous chapter considered the problem of meaning representation. Complex structured data was converted into semantic triples so that the information could be expressed in a compact, standardized, and machine-interpretable form. The problem considered in this chapter is the second stage of the data-to-text conversion: given the information contained in a set of triples, how should it be said in natural language?

This stage is called *surface realization*. Let $T = \{(s_i, p_i, o_i)\}_{i=1}^n$ denote a set of subject–predicate–object triples and let y denote a natural-language text. A surface realizer is a mapping

$$G : T \longrightarrow y, \tag{4.1}$$

which must express the facts in T fluently and coherently. The mapping is not limited to replacing predicates with fixed phrases. It can require entity ordering, aggregation of related facts, selection of referring expressions, realization of numerical values, and the choice of a suitable sentence structure. The generated text should preserve the information in the triples while avoiding unsupported statements and unnecessary repetition.

A large language model can be asked to perform this mapping directly using a zero-shot or few-shot prompt. Such a solution can produce fluent text, but the realization behavior is then distributed across model parameters, prompt instructions, and sampling decisions. It is consequently difficult to inspect which rules were applied, to reproduce a particular decision, or to diagnose a hallucination. A direct LLM call also makes the model part of every inference step. The approach taken here changes the role of the LLM: rather than asking it to verbalize each input instance directly, it is asked to construct a program that performs the conversion. The resulting program can be executed, tested, and inspected as an explicit surface realizer.

This idea is directly motivated by the work of Lango and Dusek [6], who describe a neurosymbolic framework in which LLM agents implement an NLG system from scratch. Their agents produce interpretable rule-based Python generators from RDF triples, without requiring in-domain human reference texts or conventional supervised training. The generated code provides a transparent representation of the realization process and can generate text using ordinary CPU execution. The work demonstrates that LLMs can be used to build an NLG system rather than only to act as the NLG system at inference time.

The distinction in this work concerns how the generator program is obtained. Lango and Dusek use collaborative interactions between multiple LLM agents to design and refine the rule-based gen-

erator. Here, Google’s AlphaEvolve idea is used instead: the LLM is placed inside an evolutionary optimization loop and proposes mutations of an existing program. The implementation used in the experiments is OpenEvolve, an open-source evolutionary coding framework that provides the AlphaEvolve-style orchestration layer.

During evolution, an LLM receives the current program together with selected alternative programs and task-specific instructions. It then proposes a modified program, which is evaluated on RDF-to-text examples. The evaluation compares generated texts with references using automatic metrics and can additionally use an LLM-based judge. The evolutionary database stores candidate programs and their measurements, making the final result not merely a generated response, but an executable and inspectable artifact.

The remainder of this chapter first introduces agentic programming and the AlphaEvolve framework. It then describes OpenEvolve, the program representation and database, candidate selection, prompt construction, LLM response handling, and the evaluation process used for surface realization.

4.2 Modern approaches for writing complex programs

Writing a complex program is more than translating a short natural-language description into a code fragment. A complete solution usually requires several decisions about decomposition, data structures, interfaces, error handling, testing, and the interaction between multiple components. These decisions are often coupled: a change to one component can invalidate assumptions made by another component. Consequently, the generation of a single locally plausible code fragment does not guarantee that a complete program is correct or useful.

Traditional program synthesis and code-completion systems address only part of this problem. Program synthesis generally searches for a program satisfying a formal specification or a collection of input-output examples, whereas code completion systems predict a local continuation of an existing source file. Both approaches can be highly effective for constrained tasks, but neither usually performs the complete iterative workflow required for a large software engineering task. Such a workflow includes requirement analysis, implementation, execution, diagnosis of failures, modification of several files, and repeated validation.

4.2.1 Agentic programming

The recent development of large language models has led to a transition from one-shot code generation towards *agentic programming*. Wang et al. [13] define this paradigm through the use of LLM-based agents that autonomously plan, execute, and refine software-development tasks while interacting with external tools. The LLM is therefore embedded in a control loop rather than being used only as a function that maps a prompt to a final source fragment. The agent can inspect the workspace, formulate intermediate goals, edit files, run tests or compilers, interpret the resulting diagnostics, and decide what action should be taken next.

An agentic programming system can be characterized along several related dimensions. It can be reactive, responding to a user request or a tool result, or proactive, selecting subsequent actions in pursuit of a longer-term goal. It can operate in a single turn or across multiple turns, and it can be standalone or augmented with tools such as terminals, compilers, debuggers, profilers, language servers, and version-control systems. Finally, it can be static, using the same procedure for every task, or adaptive, changing its plan in response to observations from the development environment.

4.2.2 Current state of the art

The state of the art in agentic programming is represented not by a single model architecture, but by a combination of a capable code-oriented LLM and an orchestration layer. Modern systems can use long context windows, repository retrieval, structured tool calls, shell commands, test runners, and version control operations. Multi-agent designs can assign different roles to planning, implementation, testing, code review, and documentation. In all of these cases, the central improvement over one-shot generation is the ability to use concrete feedback from the environment to revise an earlier decision.

The most important enablers of these systems are structured prompting, state and context management, tool integration, and execution monitoring [13]. Structured prompts provide the model with a role, a task decomposition, and constraints on the output. Memory mechanisms preserve plans, tool results, and previous decisions that no longer fit into the immediate context window. Tool interfaces ground the model's actions in executable operations, while execution monitoring provides an observation on which the next action can be based.

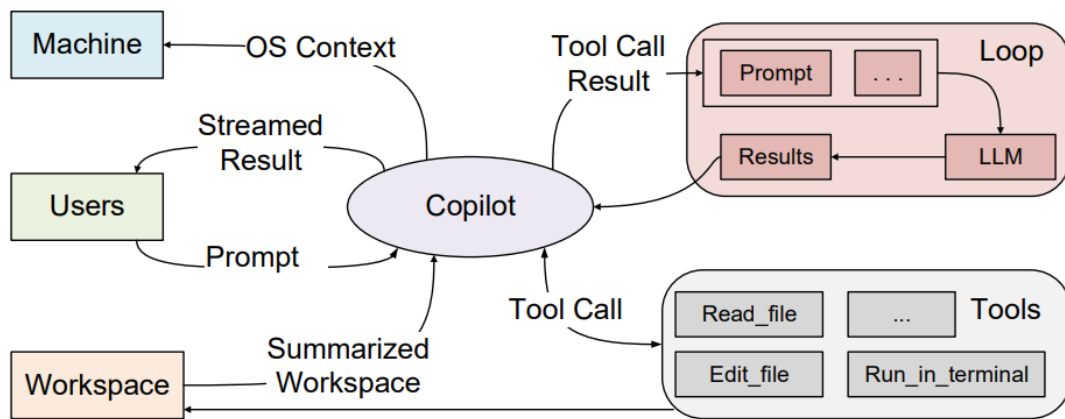


FIGURE 4.1: From Wang et al. [13] fig. 1: An example workflow of an AI coding agent.

4.2.3 Challenges and possible improvements

Despite the rapid progress, agentic programming remains limited in several ways. First, generated code is probabilistic and can contain syntactic errors, incorrect assumptions, hallucinated APIs, or changes that solve one test case while breaking another. A successful execution on a small test set is not a proof of correctness. Second, long software-engineering tasks produce more context than can be placed in a single prompt. Important decisions can be forgotten, and repeated failures can be rediscovered instead of being avoided. Third, conventional compilers and development tools expose diagnostics designed primarily for human interpretation. Their outputs do not always provide the structured causal information needed by an autonomous system to relate a failure to a particular edit.

Evaluation is another central limitation. Many existing benchmarks focus on small, self-contained tasks, a small number of programming languages, or a binary pass/fail signal. Such benchmarks do not fully measure long-horizon planning, tool-use efficiency, recovery from failure, repository-level changes, or collaboration with a human developer [13]. In addition, autonomous execution creates safety and privacy concerns: an agent may access sensitive files, execute unsafe

commands, or make changes that are difficult to inspect and reverse. Finally, repeated inference and tool calls can be expensive, especially when a large model is used for every intermediate step.

Several improvements follow from these limitations. Toolchains can expose structured diagnostics, abstract syntax trees, intermediate representations, profiling data, and explicit links between edits and failures. Context can be organized into short-term interaction history, medium-term plans, and long-term project memory, with retrieval and summarization used to control its size. Reversible edits, sandboxed execution, least-privilege tool access, and human approval for high-impact operations can improve safety. Benchmarks should also contain realistic multi-file tasks, multiple languages, tool interaction, intermediate feedback, and metrics for robustness and recovery rather than functional correctness alone.

4.3 AlphaEvolve

4.3.1 Motivation and problem formulation

AlphaEvolve is an evolutionary coding agent introduced by Novikov et al. [8]. It was designed for scientific and engineering problems in which candidate solutions can be represented as programs and assessed automatically. The motivation is that LLMs can propose creative algorithmic ideas, but an isolated LLM response is not a reliable optimization procedure. A generated program can be syntactically valid and still be slower, less accurate, or less robust than the program that preceded it. AlphaEvolve places the LLM inside an evolutionary loop in which every proposed change is grounded by execution and scoring.

This design solves a problem that is difficult for either component alone. Evolutionary search supplies selection, persistence, and a mechanism for backtracking from unsuccessful ideas, while the LLM supplies mutations that can change the program at a semantic level rather than only tuning a fixed vector of numerical parameters. The evaluator prevents an attractive explanation or a plausible-looking patch from being accepted without evidence of improved performance.

The examples reported by Novikov et al. [8] illustrate the breadth of this formulation. AlphaEvolve discovered a procedure for multiplying two 4×4 complex-valued matrices using 48 scalar multiplications, improving on the previously known 49-multiplication result in that setting. In engineering applications, it produced a matrix-multiplication kernel with an average speedup of 23% and contributed to a reported 1% reduction in Gemini’s training time. These results are not required for using the framework, but they demonstrate why evolving executable algorithms can be more useful than generating a single static answer.

4.3.2 Architecture of AlphaEvolve

The description of AlphaEvolve separates the discovery process into the following cooperating components [8]:

1. The *task specification* supplies the initial program, the evaluator, optional configuration, and any fixed domain knowledge.
2. The *prompt sampler* selects previously evaluated programs and assembles the context shown to the language model.
3. The *LLM ensemble* proposes one or more modifications to a selected program. Models with different capabilities or latency can be combined so that many inexpensive ideas and occasional more capable ideas are explored.

4. The *mutation layer* parses the model response and applies the proposed changes to the current source code. For large programs, targeted diff blocks avoid sending or rewriting unrelated code.
5. The *evaluators* execute the candidate and return its metrics. Evaluation can include cascades, parallel test cases, multiple objectives, and additional LLM-generated feedback.
6. The *program database* stores candidates and metrics and controls the balance between exploiting high-scoring programs and exploring different program structures.
7. The *distributed controller* coordinates LLM requests and evaluations asynchronously so that slow evaluations do not prevent other candidates from being processed.

Let $\mathcal{P}$ denote the space of programs that satisfy the interface of a task. The user supplies an evaluator

$$h : \mathcal{P} \longrightarrow \mathbb{R}^m,$$

which returns one or more scalar metrics. The metrics are conventionally maximized, although the evaluator can negate a quantity such as execution time when minimization is required. Starting from an initial program p_0 , the system repeatedly generates a candidate p' , evaluates it using h , and stores the resulting program together with its metrics. The database then determines which previously evaluated candidates should influence later generations.

The interaction between these components is deliberately cyclic. A program is not selected only once at the beginning of the experiment. Its score, code, and observed failures become part of the context used for subsequent mutations. The database consequently acts as an external memory for the evolutionary agent.

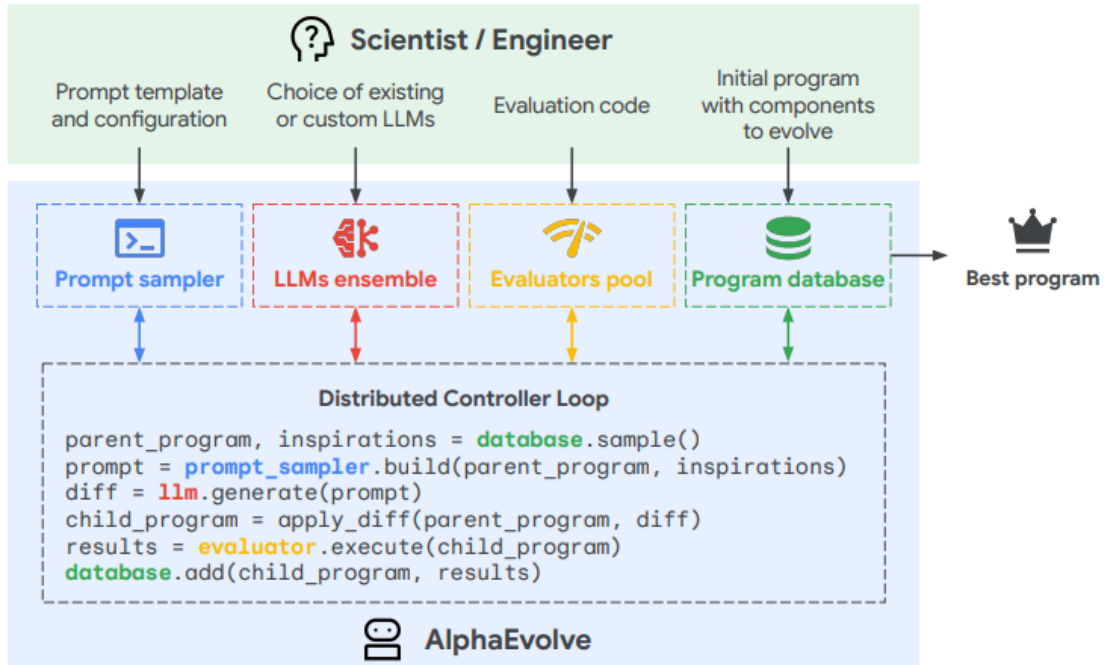


FIGURE 4.2: From Novikov et al [8] fig. 2: Expanded view of the AlphaEvolve discovery process.

4.3.3 Task specification and program representation

An AlphaEvolve task has three essential inputs. First, an initial program must be complete enough to run, even if its implementation is deliberately simple. Second, the part of the source code that may be changed is marked with `# EVOLVE-BLOCK-START` and `# EVOLVE-BLOCK-END`. The remaining code forms a fixed skeleton that defines imports, data types, and the public interface. Third, an evaluator function must be provided. The evaluator receives the generated program, executes it on the task-specific inputs, and returns a dictionary of scalar metrics.

4.3.4 Prompt sampling and creative generation

AlphaEvolve prompts contain more than the current program. Depending on the configuration, the context can contain high-scoring programs, structurally diverse programs, previous attempts, evaluator outputs, artifacts, explicit human-written instructions, and relevant examples or literature. This context allows an LLM to compare alternatives rather than repeatedly proposing a mutation without knowing what has already been tried.

The LLM is asked to make a concrete source-level change. For a diff-based mutation, the expected response is a sequence of blocks of the form

```

<<<< SEARCH
original code
=====
replacement code
>>>> REPLACE

```

The search part identifies the source segment to be replaced, and the replace part contains the candidate implementation. A complete rewrite can be used for short programs or when a targeted change is not appropriate. In both cases, the model is instructed to preserve the public interface and to optimize the metrics defined by the evaluator.

AlphaEvolve uses an ensemble rather than relying on only one generation model. In the original system, a faster model increases the number of explored ideas, while a more capable model can supply less frequent but more substantial changes [8]. The ensemble is not an ensemble in the sense of averaging source code. A single model response is selected for each mutation, after which the resulting program is evaluated independently.

4.3.5 Evaluation, evolutionary storage, and distributed execution

AlphaEvolve supports several mechanisms for making the evaluation stage both useful and affordable. An evaluation cascade can apply inexpensive tests first and run expensive tests only for candidates that pass earlier thresholds. Parallel test cases can be distributed across workers. Multiple metrics can be returned simultaneously, both because they may represent genuine objectives and because programs that perform well under different metrics can provide more varied context for future mutations. Optional LLM feedback can be used for properties that are difficult to express in a conventional evaluator, such as readability or simplicity.

The system is implemented as an asynchronous pipeline. LLM generation and evaluation are started whenever their required inputs are available, and the controller waits only at dependencies between stages. This maximizes the number of candidates evaluated within a fixed computation budget instead of minimizing the latency of an individual candidate. The resulting evolutionary database is inspired by MAP-Elites and island-based population models. MAP-Elites preserves

high-performing programs in different regions of a feature space, while islands allow partially independent populations to explore different directions before selected migration.

4.3.6 Scope and limitations of AlphaEvolve

The unique feature of AlphaEvolve is the combination of open-ended LLM mutations with machine-gradeable selection. It is not a general replacement for a software engineer, and it cannot be applied directly when candidate quality cannot be measured by an evaluator. Its results depend on the quality, coverage, and resistance to exploitation of the evaluation function. A weak metric can reward a program that optimizes the measurement rather than the intended task. The search can also require substantial computation because many candidates must be generated and evaluated, and there is no general guarantee that the best program will be found within a finite budget.

These limitations are particularly relevant to surface realization. Text quality is only partially captured by lexical metrics, while LLM judges may be inconsistent or sensitive to output formatting. The experiment therefore retains both reference-based metrics and judge feedback, logs failed cases, and keeps the mutation interface narrow enough that candidates can be evaluated repeatedly.

4.4 OpenEvolve

4.4.1 Open-source implementation

OpenEvolve is an open-source implementation of the AlphaEvolve design [10] and is used as the execution framework for all surface-realization experiments. The implementation exposes the same basic interface as AlphaEvolve: an initial program, an evaluator, and a configuration determine an iterative process in which programs are proposed, scored, and registered for future use.

The main controller loads the initial source, initializes the LLM ensemble, prompt sampler, evaluator, and program database. The initial program is evaluated before the evolutionary workers are started. On a fresh run it is then inserted into the database as generation zero. This makes the baseline available to the selection mechanism and ensures that evolution starts from a measured candidate rather than an unscored source file.

4.4.2 Execution architecture used in the thesis

The local implementation uses parallel workers for process-based parallelism. Before a worker is submitted, the controller selects a parent program and inspiration programs, which are selected the same way as the parent program, and are used as a source of inspiration for the LLM. Then the controller builds the prompt, calls the LLM, parses the response, and evaluates the resulting source.

Consequently, the logical evolution loop is:

1. select a parent and inspirations;
2. assemble a prompt from the selected context;
3. generate and parse an LLM mutation;
4. execute the candidate evaluator;
5. register the candidate, metrics, and feedback;

6. repeat until the iteration budget, target score, or stopping rule is reached.

The implementation is asynchronous in practice. Several workers can use snapshots of the program database made close together, and their results can be incorporated in a different order from the numerical iteration identifiers. This increases throughput but means that a candidate does not necessarily use the immediately preceding candidate as its parent.

4.4.3 Extensions made for the experiments

Several changes were made to the OpenEvolve’s algorithm. They address both the search distribution and the failure modes that are common when a language model is required to emit an exact source-code patch.

Fitness-weighted and island-specific selection. The database was extended with roulette-wheel selection. Parent and inspiration programs can be sampled with probabilities derived from their fitness instead of being selected uniformly. A small positive lower bound is used in the fitness weights so that low-scoring programs are not removed from consideration immediately. LLM configurations can also be assigned to islands, allowing a population to be associated with a particular model or model mixture. An optional iteration-based model schedule was added for periodically forcing a chosen model.

Configurable inspirations. The number of inspiration programs that are selected and added to the prompt can be controlled.

More robust SEARCH/REPLACE application. The original exact line matching was extended with trailing-whitespace cleanup and common-indentation handling. A model can therefore return a source block with a different outer indentation while the replacement is still inserted at the correct nesting level. If the response has no parsable diff, a corrective prompt asks the LLM to return the required format. If a syntactically valid diff does not change the source because its SEARCH block does not match, a second corrective prompt includes the parent code and asks the LLM to repair the SEARCH section.

Retention of failed candidates. Malformed or unchanged proposals are written to a separate directory instead of being silently discarded. Duplicate source code is detected before insertion. These changes prevent repeated evaluation of identical candidates and preserve evidence needed to diagnose the behavior of the LLM and the mutation parser.

Explicit zero scores for timeouts. The evaluator integration was changed so that timeout and failed-evaluation paths give a score of 0.

4.5 Program database and initial program

4.5.1 Program records and persistence

The program database is implemented in a way to store those principal fields:

- a unique identifier and the program source;
- the identifier of the parent program, generation number, timestamp, and iteration at which the program was found;

- a dictionary of evaluation metrics, including the fitness metric;
- derived values such as complexity and diversity and metadata such as island assignment and a human-readable change summary;
- optional prompts and LLM responses associated with its creation;
- optional artifacts and embeddings used for feedback and novelty checks.

The active database is maintained in memory for fast selection. For each program, feature coordinates are calculated before insertion. In the surface-realization task, code length is used as the built-in complexity feature, while diversity is approximated from source-code length, line-count, and character-set differences. Feature values are scaled and assigned to bins. The feature coordinates form a cell key, for example a pair of coordinates for complexity and diversity.

At most one program is retained as the occupant of a feature cell. If a new program maps to an empty cell, the cell is occupied. If it maps to an occupied cell, the program with the better fitness is retained as the cell occupant. The database separately stores a bounded set of elite programs. The absolute best program is tracked independently so that it is protected from ordinary population cleanup. When the configured population limit is exceeded, the lowest-fitness unprotected programs are removed from the active in-memory population, together with their references in islands and cells. The main limits are configurable.

4.5.2 Initial program

The initial program defines a small `Triple` data class with `subject`, `predicate`, and `object` fields. Its evolution block contains the following baseline behavior: `predict` receives a list of triples and returns a sentence describing the predicate, subject, and object of the first triple. This implementation is intentionally minimal. It is syntactically valid and satisfies the interface, but it ignores additional triples, treats every predicate identically, and does not model relations between entities.

The purpose of evolution is not merely to make this baseline longer. The desired search direction is from a generic fallback sentence towards a domain-aware surface realizer. Useful changes include mapping predicates to natural-language templates, grouping triples by subject, identifying a main entity, ordering attributes, joining related entities, selecting relative clauses, and maintaining grammatical agreement. These changes are evaluated by the generated text rather than being prescribed as a fixed hand-written algorithm, which allows multiple realization strategies to be explored.

4.6 Program selection and prompt engineering

4.6.1 Parent and inspiration selection

Selection is performed before a worker process is launched. For roulette selection, the parent is selected from the island using a fitness-weighted softmax distribution. The use of a distribution rather than always selecting the best program retains a non-zero probability of returning to less explored solutions.

Inspirations are selected separately from the parent. They are alternative programs that can provide a design idea without being the source that is directly modified. With roulette, their probabilities are also based on fitness, and the parent itself is excluded from the candidate inspiration

set. The selected parent and inspirations are passed to the worker, which reconstructs their source code from the database.

4.6.2 Prompt composition

The system message used for the surface-realization experiment contains four kinds of fixed information:

- the role of the model as an expert data engineer;
- the domain and the required `predict` function signature;
- an example in which several triples are combined into one complex sentence;
- the list of predicates that may occur, with one example triple for each predicate.

The user message is assembled for each selected parent. Its main parts are:

- the current metrics and fitness score;
- optional evaluator artifacts, such as low-scoring triples and the generated text that caused them;
- the complete current source code;
- the mutation task, including the requirement to improve the score, preserve the interface, and use the exact `SEARCH/REPLACE` format.

Consequently, the code and score of the selected parent are shown directly in the current-program section. An inspiration is rendered separately with its own score and source code. A high-scoring program can therefore provide a strong implementation pattern, while a structurally different inspiration can suggest a different way of grouping or ordering triples.

4.6.3 `SEARCH/REPLACE` instructions and their extensions

The mutation prompt asks for a targeted edit rather than a complete rewritten file. The `SEARCH` section must identify source text present in the current program, and the `REPLACE` section must contain the proposed replacement. The controller extracts all matching blocks with a regular expression, applies them line by line, and checks whether the resulting source differs from the parent. This format makes changes auditable and reduces the risk that the LLM rewrites fixed parts of the program unintentionally.

The format is also a significant source of failure. A model can describe a patch in prose, omit one of the delimiters, copy an outdated version of the source, or use an indentation that prevents an exact match. These problems are addressed in two ways. First, trailing whitespace is normalized and common indentation is removed while matching, after which the replacement is re-indented at the original location. Second, malformed responses and unapplied responses are sent to dedicated repair prompts. These safeguards preserve the replacement text where possible and focus the subsequent LLM call on producing a matchable `SEARCH` section.

4.7 Large Language Model

4.7.1 Parsing the LLM response

OpenEvolve uses an OpenAI-compatible client interface, so the same orchestration code can communicate with hosted models or local servers. The response is processed in the worker in the following order:

1. The response is searched for one or more SEARCH/REPLACE blocks.
2. Each block is normalized into source lines and applied to the parent source.
3. If at least one block is found but no source line is changed, the response is treated as an unapplied mutation rather than as a successful no-op.
4. A new identifier is assigned and the candidate is passed to the evaluator.

When no valid block is found, the response is not immediately treated as a failed iteration. Up to four attempts are made. The first repair instruction explains the required delimiter format. If a block was parsed but could not be applied, the repair instruction additionally contains the original parent code and asks the model to correct the SEARCH section while preserving the replacement. The intermediate invalid responses are joined and retained as diagnostic feedback.

If all repair attempts fail, a child record carrying the unchanged parent code is returned with an error and a `no_valid_diffs` or `no_changes` artifact. The controller writes this record to the list of broken programs and does not add it to the active evolutionary population.

OpenEvolve also contains a full-rewrite mode. When diff-based evolution is disabled, a fenced code block or the plain response is parsed as the complete candidate source. This mode was retained for compatibility, but the surface realization experiments use targeted diffs because the evolved function is embedded in a fixed program skeleton.

4.7.2 Invalid programs and execution safety

A generated program can fail before it produces any text. Dynamic loading can fail because of a syntax error or missing import; the required `predict` function can be absent or have the wrong signature; and a call can return a non-string value, raise an exception, or run indefinitely. These cases are converted into zero-valued fitness and artifacts describing the failure rather than terminating the entire evolution run. The artifacts can then be included in a later prompt, where they provide a concrete example of what should be avoided.

4.8 Evaluation

4.8.1 Evaluator integration

The OpenEvolve **Evaluator** is a generic wrapper around the task-specific evaluation file. For every candidate, it invokes the function named `evaluate`. The result contains metrics and artifacts. The wrapper applies the configured timeout, handles retries, converts both result formats to a common representation, and collects pending artifacts for the program record.

The initial program is passed through the same evaluator before evolution starts. This establishes the baseline against which later candidates are compared. A worker follows the same path after it has applied an LLM mutation. The task-specific evaluator is therefore the boundary

between source-code generation and evolutionary selection: OpenEvolve does not infer whether a sentence is good from the source code itself.

4.8.2 Reference data and candidate execution

The surface-realization evaluator loads a supplied reference data corpus. Each selected entry contains a triple set and one or more reference lexicalizations.

Before the examples are processed, the evaluator loads the candidate module dynamically and checks that `predict` exists and accepts exactly one argument. For each reference data entry, the list of triples is passed to `predict`. A result is accepted only when it is a string. An empty string receives a per-example zero lexical score. Exceptions, invalid return types, and timeouts terminate the affected evaluation path and produce zero-valued metrics with an explanation in the artifacts.

4.8.3 Reference-based scoring

For every non-empty generated text, the evaluator computes a BLEU score against all reference lexicalizations associated with the entry. The average over the selected domain entries is returned as an aggregate reference-based measure.

BLEU value is used as `combined_score`, and it is the quantity used by the evolutionary database for fitness comparisons.

4.8.4 LLM as a judge

(mention here about themis and what it is. tell that instead of themis model we can use a different model as a judge)

When Themis is enabled, the evaluator constructs an additional judge prompt for each non-empty candidate output. The prompt contains the input triples and the generated text and asks the judge to assess two criteria: whether the text faithfully expresses the triples and, where possible, whether the information is expressed as one complex sentence instead of several disconnected sentences.

A rating on a scale from 1 to 5 is averaged over the examples and divided by 5. When Themis is available, this normalized judge score is the combined fitness, while the average BLEU value is retained as a diagnostic metric. Thus the judge can favor a faithful and compact realization even when it uses wording that has limited n-gram overlap with the references.

4.8.5 Artifacts as evaluator feedback

The evaluator returns low-scoring examples as artifacts. A Themis artifact contains the input triples, the generated text, the judge review, and the score. If too many examples qualify, at most three are sampled for storage and prompt rendering. In a run without Themis, low BLEU examples contain the references as well. On the next mutation, these artifacts are rendered under the feedback section of the user prompt. The resulting feedback loop is therefore more specific than a scalar fitness value: the LLM can see which triples were omitted, which relation was expressed poorly, or which output was split into several sentences.

The evaluator distinguishes metrics from artifacts. Metrics are numeric values used by the database, whereas diagnostic strings, reviews, traces, and error messages are not included in the fitness calculation. This separation prevents an explanatory text from accidentally changing the ranking while still making the explanation available to the next generation.

4.8.6 Registration and continuation of the loop

After evaluation, the worker creates a child **Program** containing the candidate source, parent identifier, generation, iteration, metrics, and a change summary. The database then calculates feature coordinates, updates the corresponding MAP-Elites cell and the archive, enforces the population limit, and updates the global and island-specific best-program references. Artifacts and prompts are stored, and the evolution tracer records the parent, child, metrics, source code, response, and improvement delta when tracing is enabled. A new iteration is then submitted, so the evaluated child can influence later parent and inspiration selection.

This ordering is the central feedback mechanism of the method. A score without registration would not affect the search, and a database without evaluation would have no objective basis for selection. The repeated interaction between the two is what converts LLM code generation into an evolutionary surface realization procedure.

Chapter 5

Experiments

5.1 Meaning representation

5.1.1 Data Collection

The experimental evaluation in this work requires structured data from diverse domains. To obtain such data, *quintd* [3] is used, a framework for collecting *ad hoc* data-to-text datasets. Quintd downloads up-to-date structured data from real-world APIs. The framework supports five domains: weather forecasts, mobile phone specifications, ice hockey game summaries, health statistics, and world knowledge entities.

From the five domains available in *quintd*, four datasets are used in the experiments:

GSMarena Mobile phone specifications scraped from the GSMarena website. Each instance describes a single phone model with nested categories (Network, Launch, Body, Display, Platform, Memory, Camera, Battery, etc.) containing attribute–value pairs.

OpenWeather Five-day weather forecasts obtained from the OpenWeather API. Each instance contains a time series of 40 three-hour intervals, with each interval recording temperature, humidity, wind speed, cloud coverage, and weather conditions.

OWID (Our World in Data) Country-level time series metrics from the Our World in Data project. Each instance represents a single metric (e.g., life expectancy, vaccination rates, reproduction rate) for a specific country, containing yearly data points spanning several decades.

Wikidata Knowledge graph entities and their properties from Wikidata. Each instance describes an entity (e.g., a country, film, or organization) with a list of property–value pairs (e.g., capital, cast member, language used).

The fifth domain, *ice hockey*, is available in *quintd* but is excluded from our experiments due to API rate limits and associated costs.

All datasets are provided as JSON files containing arrays of structured records. A *raw structured instance* is defined as a single JSON object representing one self-contained data record from any of these domains. Each instance contains nested fields with structured data—time series, categorical values, numerical measurements, or hierarchical attribute lists—that must be converted into semantic triples. Let $D = \{d_1, \dots, d_N\}$ denote the set of N such instances extracted from the input JSON for a given domain.

Large language model. All LLM invocations across the three methods use *Gemma 4 31B IT* [11, 12], a 31-billion-parameter instruction-tuned language model released by Google. Gemma 4 uses a hybrid attention architecture: most layers employ sliding-window attention with head dimension 256, while a subset of layers use global attention with head dimension 512. The model is served via *vLLM* [5], a high-throughput inference engine that uses *PagedAttention* to efficiently manage KV-cache memory, enabling batched inference with dynamic sequence lengths.

5.1.2 Training Configuration

The base model for all fine-tuning experiments is *Gemma 4 31B IT* (described in Section 3.2).

All training is performed on NVIDIA A100-SXM4 GPUs with 40 GB of VRAM, using the PLGrid Athena cluster. Table 5.1 summarizes the complete training configuration.

TABLE 5.1: QLoRA training configuration

Component	Value
Base model	Gemma 4 31B IT
LoRA rank (r)	16
LoRA scaling (α)	32
LoRA dropout	0.05
Target modules	420 (all attention and MLP projections)
Trainable parameters	25M (<0.1% of base model)
Quantization	4-bit NF4 with double quantization
Compute precision	bfloat16
Optimizer	Paged AdamW 8-bit
Learning rate	10^{-4} (cosine schedule)
Warmup ratio	0.03
Epochs	3
Effective batch size	16 (per-device: 1, accumulation: 16)
Max sequence length	4096–8192 (domain-dependent)
Loss type	Completion-only (prompt masked)

The LoRA configuration targets all attention and MLP projection matrices in the language model backbone (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`), using a regular expression that scopes the adapter placement to the language model sub-network and excludes the vision and audio towers present in the multimodal Gemma 4 checkpoint. This yields approximately 420 target modules and roughly 25 million trainable parameters—less than 0.1% of the base model.

The maximum sequence length is adjusted per domain to accommodate the varying sizes of the raw structured instances while staying within the GPU memory budget. Wikidata instances are small (at most a few hundred bytes), so a maximum length of 8192 tokens is used. GSM Arena and OWID instances are larger, and a maximum length of 4096 tokens covers the majority of examples. OpenWeather forecast instances are the largest (approximately 18 KB each), and a maximum length of 6144 tokens is used to retain the bulk of each example while remaining within the activation memory ceiling. Instances exceeding the maximum length are truncated.

After training, the LoRA adapter weights are combined with the frozen base model weights to produce a single, standalone model. This process, known as *weight merging*, computes $W_{\text{final}} = W_0 + \Delta W$ for each adapted layer, effectively absorbing the adapter’s learned modifications into the base model. The resulting model contains no separate adapter components and can be served as a standard `bfloat16` checkpoint. The merged model is a drop-in replacement for the base model: it can be served by the same vLLM inference engine used by the tripler pipeline, with no adapter-loading overhead at inference time. The result is a domain-specialized model that, given

a raw structured instance, produces both a natural-language verbalization and the corresponding set of semantic triples in a single decode pass.

5.2 Surface realization

5.2.1 Illustrative evolution trace

The completed Building run in the `5_1krand30m4p1themis/Building_output/` directory shows the complete data flow. It was configured for 1,000 evolutionary iterations, used four worker processes, one island, a population limit of 50, an archive of 10, one inspiration per prompt, and enabled Themis judging and JSON evolution tracing. The output contains 948 ordinary program records, 15 records in `broken_programs/`, and 896 trace entries. The counts differ because several iterations produced malformed or duplicate candidates and because persistent program files document historical candidates rather than only the final active population.

The initial program was evaluated at iteration zero with a recorded `combined_score` of approximately 0.0300 and a BLEU diagnostic of approximately 0.1871. The first accepted mutation replaced the one-line fallback with a predicate map, an empty-input guard, subject grouping, and multi-triple joining. Its recorded combined score increased to approximately 0.0358, while its BLEU diagnostic increased to approximately 0.4295. Later mutations introduced linked-subject traversal and relative-clause selection. The best recorded candidate was found at iteration 794, generation 10, after a mutation that simplified subject processing and removed quote and angle-bracket delimiters from object values. It had a recorded combined score of approximately 0.0676. These values illustrate the mechanics of the evolution trace and are not held-out test results; the latter are produced separately by `final_test.py`.

The online evaluator uses WebNLG training and development entries to provide fitness during evolution. The final-test script instead loads the held-out WebNLG test file and computes BLEU, METEOR, SENLEN, BLEURT, judge scores, grammaticality, omission, and addition measures. Keeping these two paths separate makes it possible to use the evolutionary score to drive search while reporting generalization in the experiments chapter.

5.3 Whole system evaluation

Chapter 6

Discussion

Chapter 7

Summary

Bibliography

- [1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. <https://arxiv.org/abs/2305.14314>, 2023.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. <https://arxiv.org/abs/2106.09685>, 2021.
- [3] Zdeněk Kasner and Ondřej Dušek. Beyond traditional benchmarks: Analyzing behaviors of open llms on data-to-text generation. <https://arxiv.org/abs/2401.10186>, 2024.
- [4] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. <https://arxiv.org/abs/1412.6980>, 2014.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. <https://arxiv.org/abs/2309.06180>, 2023.
- [6] Mateusz Lango and Ondrej Dusek. LLM agents implement an NLG system from scratch: Building interpretable rule-based RDF-to-text generators. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 2025–2040, Suzhou, China, 2025. Association for Computational Linguistics.
- [7] Sourab Mangrulkar, Sylvain Gugger, Lysak Lysak, et al. PEFT: Parameter-efficient fine-tuning of billion-scale models on low-resource hardware. <https://github.com/huggingface/peft>, 2022.
- [8] Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. <https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/AlphaEvolve.pdf>, 2025.
- [9] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Ray, et al. Training language models to follow instructions with human feedback. <https://arxiv.org/abs/2203.02155>, 2022.
- [10] Asankhaya Sharma. Openevolve: An open-source evolutionary coding agent. <https://github.com/algorithmicsuperintelligence/openevolve>, 2025.
- [11] Gemma Team et al. Gemma: Open models based on gemini research and technology. <https://arxiv.org/abs/2403.08295>, 2024.
- [12] Gemma Team et al. Gemma 4 technical report. <https://arxiv.org/abs/2607.02770>, 2026.
- [13] Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. Ai agentic programming: A survey of techniques, challenges, and opportunities. <https://arxiv.org/abs/2508.11126>, 2025.

Appendix A

Prompts

A.1 Text-First Extraction with Normalization

This section contains the complete text of all system prompts used in the Text-First Extraction with Normalization method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: `{"text":"..."}`. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Triples-From-Text System Prompt (P_{tr})

You extract semantic triples from natural language text. Return ONLY JSON with this exact schema: `{"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}`. Rules: preserve the full meaning of the input text, avoid duplicate triples, and keep predicates in lower_snake_case when possible.

Candidate-Pairs System Prompt

You find candidate predicate pairs that may be semantically equivalent. Return ONLY JSON with schema: `{"pairs":[{"predicate_a":"...", "predicate_b":"..."}]}`. Rules: include only pairs with high likelihood of strict semantic equivalence; do not include pairs that are merely related; do not include duplicates or self-pairs.

Strict-Equivalence System Prompt

You decide if two predicates are semantically equivalent in meaning. Return ONLY JSON with schema: `{"equivalent": true|false, "confidence": 0.0-1.0, "reason": "..."}`. Be strict: equivalent only if they can be safely merged in a knowledge graph.

A.2 Blueprint-Iterative Refinement

This section contains the complete text of all system prompts used in the Blueprint-Iterative Refinement method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."} . Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Blueprint-Creation System Prompt ($P_{\text{blueprint}}$)

You design extraction rules for a domain. Return ONLY JSON with schema: {"predicates":["..."], "example_triples":[{"subject":"...", "predicate":"...", "object":"..."}], "example_text":"...", "guidelines":["..."]}. Rules: include precise and reusable predicates, provide concrete example triples with strict formatting, provide an example text matching those triples, and give practical transformation guidelines.

Blueprint-Feasibility System Prompt (P_{feas})

You evaluate whether given extraction rules are sufficient to extract triples from a text. Return ONLY JSON with schema: {"possible":true|false, "reason":"...", "rule_gaps":["..."]}.

Blueprint-Revision System Prompt (P_{rev})

You revise extraction rules so triples can be produced from a text. Return ONLY JSON with schema: {"predicates":["..."], "example_triples":[{"subject":"...", "predicate":"...", "object":"..."}], "example_text":"...", "guidelines":["..."]}. Keep existing high-quality rules, but add or adjust what is necessary to cover the failing case.

Triples-From-Text-With-Blueprint System Prompt ($P_{\text{tr\&R}}$)

You extract semantic triples from natural language text using provided rules. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Follow rule guidelines and mimic the formatting style from example triples. Prefer predicates from the rule predicate list whenever possible.

A.3 Evolving Catalog

This section contains the complete text of all system prompts used in the Evolving Catalog method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."} . Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

First-Instance System Prompt (P_{first})

You extract semantic triples from text. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Use concise predicate labels in lower_snake_case when possible and avoid duplicates.

Catalog-Guided System Prompt (P_{cat})

You extract semantic triples from text using a predicate catalog with examples. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Use only predicates from the provided catalog when possible. Introduce a new predicate only when none of the catalog predicates can describe the fact.

Appendix B

Prompts for Fine-tuned Model

System Prompt for Joint Text and Triple Generation

You convert one structured data instance into concise natural language and a corresponding set of RDF semantic triples. Return ONLY JSON with schema: {"text":"...", "triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Capture the most important information, trends, extremes, and notable conditions. Use concise predicate labels in lower_snake_case when possible and avoid duplicates. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

User Message Template

Create a concise natural-language summary of this ONE data instance and extract its semantic triples.

instance_context={serialized JSON instance}

Return JSON only.

Assistant Message (Expected Output)

```
{"text": "<generated natural language text>", "triples": [{"subject": "...", "predicate": "...", "object": "..."}, ...]}
```



© 2026 Dominik Maćkowiak

Poznań University of Technology
Faculty of Computing
Institute of Computing Science

Typeset using L^AT_EX in Computer Modern.

BibT_EX:

```
@mastersthesis{ key,  
  author = "Dominik Maćkowiak",  
  title = "{Interpretable methods for data-to-text generation}",  
  school = "Poznań University of Technology",  
  address = "Poznań{\n}, Poland",  
  year = "2026",  
}
```